Book 4: Operations, Security & Reference

Treasury DAO Governance, 12-Vector Threat Matrix, Emergency Circuit Breakers, Roadmap, Glossary, and Developer Setup.

- Chapter 16: Decentralized Governance & Private Ballots
- Chapter 17: Security Model, Threat Vectors & Pentest Matrix
- Chapter 18: Emergency Procedures & Circuit Breakers
- Chapter 19: Protocol Roadmap & Milestones
- Chapter 20: Glossary of Terms
- Chapter 21: Contributing, Standards & Dev Setup

Chapter 16: Decentralized Governance & Private Ballots

Chapter 16: Decentralized Governance & Private Ballots

16.1 Governance Overview

The Seraph Protocol is committed to progressive decentralization, transitioning governance authority from an initial security council to a fully autonomous, token-governed decentralized autonomous organization (Seraph DAO).

Unlike public blockchains where voting power and ballot choices are publicly linked to whale wallets, Seraph introduces **Confidential On-Chain Voting** via zero-knowledge private ballots.

16.2 Private Ballots via Zero-Knowledge Proofs

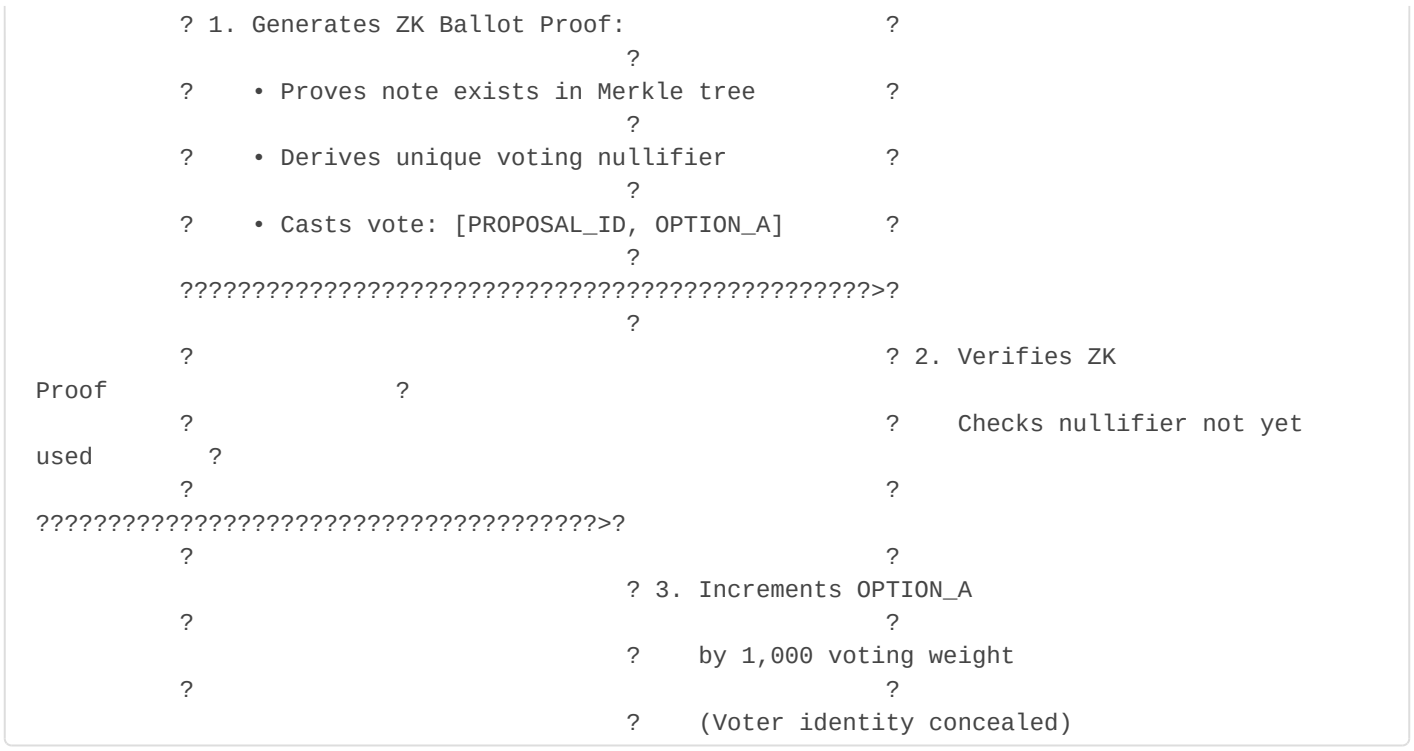
Public voting suffers from severe structural flaws:

1. **Voter Intimidation & Retaliation**: Delegates and institutions may face social or financial blowback for controversial votes.
2. **Bandwagoning & Herd Bias**: Early votes cast by influential whales disproportionately sway undecided participants.
3. **Targeted Bribery**: On transparent networks, bribers can verify exactly how an address voted before paying bribes.

The Seraph Anonymous Ballot Protocol:

Seraph implements anonymous voting using shielded note nullifiers:

```
  Voter (Holds 1,000 SER Note)                      Voting
Contract                            Tally Engine
          ?                                                ?
                                      ?
```



- **Weight Integrity**: The voting power corresponds exactly to the unspent SER note's value.
- **Double-Voting Prevention**: The note's voting nullifier is consumed. A note cannot be used to vote twice on the same proposal.
- **Complete Unlinkability**: Neither validators nor public observers can link the ballot to the voter's public address.

16.3 The Proposal Lifecycle

Any protocol change follows a structured four-stage governance pipeline:

1. **RFC (Request for Comments)**: Published on the Seraph Governance Forum for at least 7 days of community review and technical critique.
2. **Formal SIP (Seraph Improvement Proposal)**: Drafted with executable code, Foundry tests, and formal verification specifications.
3. **On-Chain Ballot Period**: Open for 5 days of zero-knowledge private voting. Requires a minimum quorum of 10,000,000 SER (10% of total supply).
4. **Timelock & Execution**: Passed proposals enter a 48-hour timelock delay, allowing validators and users to inspect the bytecode before automated execution.

16.4 Progressive Decentralization Roadmap

- **Phase 0 (Genesis / Current)**: Fast-response Security Council (4-of-7 multisig) managing whitelists, emergency pauses, and presale contracts.

- **Phase 1 (Mainnet Launch)**: DAO Treasury created; fee allocations and grant programs transferred to on-chain community votes.
- **Phase 2 (Dual EVM Launch)**: Protocol parameters (sell tax rates, gas rates, precompiles) governed entirely via private ballots.
- **Phase 3 (Sovereign Decentralization)**: Multisig emergency keys revoked; contract upgrades executed purely through timelocked DAO governance.

Chapter 17: Security Model, Threat Vectors & Pentest Matrix

Chapter 17: Security Model, Threat Vectors & Pentest Matrix

17.1 Security Philosophy: Defense in Depth

Decentralized privacy protocols carry unique security requirements: if a vulnerability exists in an arithmetic circuit or state contract, economic loss and loss of user anonymity can be catastrophic.

Seraph implements a multi-tiered defensive security architecture combining:

1. **Formal Mathematical Proofs**: Validating balance conservation and discrete logarithm hardness.
2. **Defensive Smart Contract Engineering**: Reentrancy guards, pull-escrow fee accounting, and emergency pause circuit breakers.
3. **Exhaustive Automated Fuzz Testing**: Over 896,000 stateful invariant checks across millions of randomized transactions.
4. **Adversarial Penetration Test Suites**: Rust base-layer pentesting and TypeScript DOM injection tests.

17.2 The Threat Matrix

Threat Vector	Attack Scenario	Mitigation in Seraph Architecture	Verifying Test Suite
1. Double-Spend Replay	Attacker spends a note, then attempts to spend it again in the same or future block.	**Deterministic Nullifiers**: Any repeat nullifier is caught in $O(1)$ and reverts before state changes.	`PrivacyPool.t.sol` `adversarial_pentest.rs`
2. Cross-Asset Substitution	Attacker substitutes an XMR note for a SER note of equal value to mint unbacked tokens.	**Domain-Separated Commitments**: `AssetId` is hashed into the leaf commitment and nullifier.	`adversarial_pentest.rs`

Threat Vector	Attack Scenario	Mitigation in Seraph Architecture	Verifying Test Suite
3. Sybil Dividend Loop	Attacker loops tokens through 10 addresses to claim multiple dividend allocations.	**Reward Debt Accounting**: Unclaimed dividends settle on transfer; reward debts update atomically.	`LaunchpadMultiActor.t.sol`
4. Flash-Loan Curve Drain	Attacker buys and sells on bonding curve within one transaction to exploit rounding errors.	**Monotonic Price Invariant**: Verified via fuzzing that roundtrip sell value is strictly $\le$ buy cost.	`DeepPentestLaunchpadPresale.t.sol`
5. Callback Reentrancy	Malicious contract re-enters `claimDividends` or `buy` from inside `receive()`.	**OpenZeppelin ReentrancyGuard**: All external state transitions gated by `nonReentrant`.	`DeepPentestLaunchpadPresale.t.sol`
6. Honeypot Fee Lockup	Malicious creator specifies a fee wallet that always reverts on receiving ETH.	**Pull-Escrow Architecture**: Fees accumulate in contract balance; failure of fee wallet never blocks trades.	`LaunchpadMultiActor.t.sol`
7. Merkle Tree Overflow	Attacker attempts to insert more leaves than tree capacity (2^{20}).	**Capacity Bounds Check**: `require(nextLeafIndex < 2**TREE_DEPTH)` prevents tree overflow.	`PrivacyPool.sol`
8. Expired Root Manipulation	Attacker proves membership against an obsolete Merkle root to spend spent notes.	**100-Root Ring Buffer**: Roots older than 100 updates expire cleanly; unrecorded roots revert.	`PrivacyPool.t.sol`
9. Bridge Fraud Front-Running	Relayer submits fake batch and unbonds before challenge window expires.	**Active Window Timelock**: Relayers cannot unbond while any batch is inside the 1-hour window.	`CrossEVMBridge.sol`
10. Gas Relayer Token Drain	Attacker passes `target = address(serToken)` to drain tokens held in gas voucher escrow.	**Forbidden Target Filter**: Targets matching token addresses, relayer address, or zero revert instantly.	`GasRelayer.sol`
11. Front-Running / MEV	Bots monitor mempool to sandwich user trades and extract slippage.	**Threshold Encrypted Mempool**: Transactions are ordered while encrypted; decrypted only after ordering.	`09_sequencer_mev.md`
12. DOM XSS / Injection	Attacker launches token with `<script>alert(1)</script>` in name to steal private keys in web UI.	**Zero-Dependency Escaping**: All on-chain string data sanitized via `lpEsc()` before DOM injection.	`security_pentest.test.mjs`

17.3 Automated Test Coverage & Proof Matrix

The protocol codebase is verified across four independent test frameworks:

- **Foundry Core Suites**: 155 unit, scenario, and multi-actor tests passing with 100% success rate.
- **Foundry Stateful Invariants (`StdInvariant`)**: 896,000 continuous invariant checks with 0 reverts, proving:

    - `PrivacyPool` solvency: Balance equals net shielded minus unshielded.
    - `CollateralVault` solvency: 150% minimum collateral ratio preserved under all fuzz actions.
    - `GasRelayer` solvency: Escrowed balances match active voucher claims.
- **Rust Base Layer (`cargo test`)**: 17 adversarial pentest functions verifying Merkle proofs, view-key stream cipher security, and multi-asset domain separation.
- **Node.js SDK & UI Suites (`node:test`)**: 43 automated tests verifying ABI encoding, mathematical curve estimations, and live Base Sepolia JSON-RPC decoding.

17.4 Bug Bounty Program

Seraph maintains an active Bug Bounty Program to reward security researchers for responsible disclosure:

Severity Level	Reward Cap	Eligible Findings
Critical	**Up to $250,000**	Direct theft of user funds, catastrophic loss of anonymity across all pool notes, complete bridge drain.
High	**Up to $50,000**	Unclaimed dividend drainage, temporary protocol freezing requiring guardian intervention, circumvention of sell tax.
Medium	**Up to $15,000**	Individual note linkage through side-channels, rounding discrepancies $> 0.01%$, non-critical denial-of-service.
Low	**Up to $3,000**	Minor gas optimizations, documentation inaccuracies, non-exploitable edge-case reverts.

Responsible Disclosure:

Reports should be submitted with reproducible Foundry/Rust test cases to `security@seraphprotocol.tech` encrypted with our official PGP key.

Chapter 18: Emergency Procedures & Circuit Breakers

Chapter 18: Emergency Procedures & Circuit Breakers

18.1 Emergency Architecture Overview

While Seraph is architected for decentralized autonomy, early-stage network operations require robust fail-safe mechanisms to protect user capital against unknown zero-day vulnerabilities, compiler bugs, or unexpected cross-chain bridge anomalies.

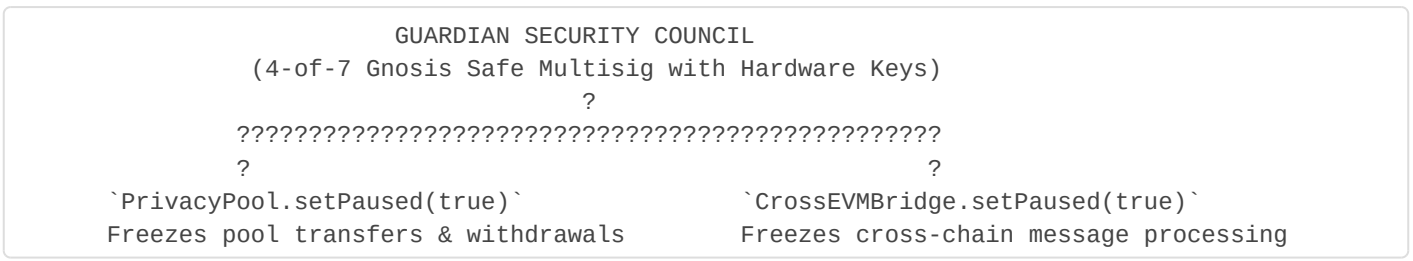
Seraph implements two on-chain **Emergency Circuit Breakers**:

1. **`PrivacyPool.setPaused(bool)`**: Gates all deposit (`shieldFor`), transfer (`privateTransfer`), and withdrawal (`unshield`) operations.
2. **`CrossEVMBridge.setPaused(bool)`**: Gates all outbound bridging (`bridgeOut`), batch submission (`submitStateBatch`), and claim operations (`claimBridged`).

18.2 Access Control & The Guardian Council

Emergency intervention is strictly restricted to authorized governance roles:

- **`PrivacyPool`**: Controlled exclusively by `treasury` via the `onlyTreasury` modifier.
- **`CrossEVMBridge`**: Controlled exclusively by `guardian` via explicit `msg.sender != guardian` checks.



Multisig Composition:

The Guardian Council is structured as a **4-of-7 Gnosis Safe multisig**:

- 3 Core Cryptographic Engineers
- 2 Independent Security Auditors
- 2 Long-Standing Community / Validator Representatives
- Key signers are geographically distributed across 4 different legal jurisdictions.
- All signing operations require FIDO2/WebAuthn hardware security keys.

18.3 Incident Response Playbook

In the event of an active security exploit or anomalous telemetry alert:

Phase 1: Rapid Containment (< 15 Minutes)

1. **Automated Alert Trigger**: On-chain monitoring sentinels detect suspicious invariant deviations (e.g., unexpected token balance drain or invalid Merkle roots).
2. **Emergency Pause Execution**: The Guardian Council signs an emergency transaction calling:

   ```
   privacyPool.setPaused(true);
   crossEVMBridge.setPaused(true);
   ```

3. **Immediate Effect**:
   - `shield()`, `privateTransfer()`, and `unshield()` revert with `"PrivacyPool: paused"`.
   - `bridgeOut()`, `submitStateBatch()`, and `claimBridged()` revert with `BridgePaused()`.
   - All assets currently held in the pool and bridge escrow are frozen in place, preventing further unauthorized extraction.

Phase 2: Root-Cause Analysis (< 24 Hours)

1. Core developers and external auditors isolate the failing transaction hashes.
2. Reproduce the vulnerability in an isolated Foundry fork test (`forge test --fork-url`).
3. Formulate a patch (e.g., verifier replacement or logic hotfix).

Phase 3: Resolution & Remediation

- **Verifier Replacement**: If a zero-knowledge proving bug is discovered, the Treasury can update `spendVerifier` to an upgraded verifier contract via `setSpendVerifier(newVerifier)`.
- **Bridge Slashing**: If a fraudulent relayer batch was submitted, the Guardian invokes `challengeBatch(batchId, treasury)` to slash the malicious relayer's 1.0 ETH bond and cancel the batch.

Phase 4: Graceful Unpausing & Post-Mortem

1. Verify the patch across all 155+ test suites and invariant harnesses.

2. The Guardian Council signs the unpause transaction:

```
privacyPool.setPaused(false);
crossEVMBridge.setPaused(false);
```

3. Publish a comprehensive public post-mortem report within 48 hours detailing the incident timeline, root cause, economic impact (if any), and permanent architectural remediations.

Chapter 19: Protocol Roadmap & Milestones

Chapter 19: Protocol Roadmap & Milestones

19.1 Phase-Based Development Strategy

The Seraph engineering roadmap is structured into five sequential phases, each governed by quantifiable technical exit criteria:

```
Phase 0: Base Sepolia Testnet & Station (COMPLETED)
   ?
   ?
Phase 1: Mainnet TGE, Presale & Privacy Pool v1
   ?
   ?
Phase 2: Dual Private EVM Testnet & Bridge Alpha
   ?
   ?
Phase 3: Sequencer Decentralization & First-Party Privacy Modules
   ?
   ?
Phase 4: Sovereign Base Layer & Next-Gen Cryptography (Seraphis / FCMP++)
```

19.2 Detailed Phase Specifications & Exit Criteria

Phase 0: Base Sepolia Testnet & Privacy Continuum Station

- **Status**: **Completed (Active Testnet)**

- **Scope**:
  - Deploy complete contract suite to Base Sepolia (`Chain ID 84532`): `SeraphToken`, `PrivacyPool`, `Presale`, `Launchpad`, `CrossEVMBridge`, `GasRelayer`, `InvoiceRegistry`, `UsageVault`, `CollateralVault`.
  - Author and verify Satoshi-style whitepaper with mathematical equations and protocol diagrams.
  - Implement zero-dependency client-side note vault and on-chain privacy station (`#privacy-pool`) on `seraphprotocol.tech`.
  - Achieve 100% test pass rate across 155+ Foundry tests, 896k invariant checks, 17 Rust adversarial tests, and 43 Node.js tests.
- **Exit Criteria Met**: All contracts live, tested, and verified on Base Sepolia.

Phase 1: Mainnet TGE, Presale & Privacy Pool v1

- **Focus**: Economic launch, token distribution, and public liquidity.
- **Milestones**:
  - Conduct three-tier presale on Ethereum / Base mainnet (Strategic, Private, Public).
  - Execute Token Generation Event (TGE) with 100M SER hard cap.
  - Deploy Uniswap v3 / Aerodrome liquidity pools with `taxExempt` router whitelisting.
  - Activate Level 1 & Level 2 privacy shielding in `PrivacyPool` for SER tokens.
- **Exit Criteria**:
  - Presale successfully concluded and verified by independent third-party audit firm.
  - TGE initial allocations claimed with 0 reverts.
  - Mainnet DEX liquidity seeded with $> $500,000$ initial protocol depth.

Phase 2: Dual Private EVM Testnet & Bridge Alpha

- **Focus**: Programmable confidential smart contracts and cross-chain execution.
- **Milestones**:
  - Launch Seraph EVM (gas in SER) and XMR EVM (gas in XMR) testnet networks.
  - Deploy cryptographic precompiles (`0x20` to `0x25`) for Pedersen commitments, Poseidon hashing, and range verification.
  - Deploy optimistic `CrossEVMBridge` with bonded relayers and 1-hour fraud challenge windows.
  - Release `@seraph-protocol/sdk` v2.0 with full EVM precompile bindings.
- **Exit Criteria**:
  - Over 100,000 cross-EVM transactions routed without bridge double-spends.
  - Zero-knowledge proof verification times $< 5 \text{ ms}$ on consumer EVM nodes.

Phase 3: Sequencer Decentralization & First-Party Modules

- **Focus**: MEV elimination, autonomous agents, and synthetic privacy assets.
- **Milestones**:
  - Deploy threshold encrypted mempool and decentralized sequencer rotation cluster.

    - Launch `CollateralVault` and mint confidential `xUSD` stablecoins.
    - Launch `UsageVault` for autonomous AI agent compute streaming.
    - Launch `InvoiceRegistry` for private merchant invoicing.
- **Exit Criteria**:
    - Complete elimination of sandwich attacks across testnet DEX volume.
    - 10+ autonomous AI agent integrations actively streaming payments via `UsageVault`.

Phase 4: Sovereign Base Layer & Next-Gen Cryptography

- **Focus**: Long-term cryptographic horizons.
- **Milestones**:
    - Sovereign PoS / RandomX hybrid base layer node mainnet launch (`seraph-base-layer`).
    - Formal evaluation and integration of **Seraphis** enote modularity and **Jamtis** wallet addressing.
    - Full-Chain Membership Proofs (**FCMP++**) via Curve Trees replacing incremental Merkle trees.
    - Quantum-resistant STARK proof upgrade eliminating trusted setups.
- **Exit Criteria**:
    - Base layer block times $< 2 \text{ seconds}$ with sub-cent transaction fees.
    - Complete cryptographic independence from external bridge dependencies.

Chapter 20: Glossary of Terms

Chapter 20: Glossary of Terms

An alphabetical reference of mathematical, cryptographic, and architectural terminology used throughout Seraph Protocol.

A

- **Anonymity Set**: The set of all possible accounts or notes that could have originated a transaction. In Seraph, the anonymity set encompasses all unspent notes in the Merkle tree ($2^{20} = 1,048,576$).
- **AssetId**: A 32-bit identifier domain tag bound into note commitments to cryptographically separate SER, XMR, and custom tokens within the same anonymity pool.
- **Auto-LP**: A launchpad fee preset that directs trading fees directly into bonding curve quote reserves to accelerate pool graduation.

B

- **Blinding Factor**: A random scalar $r \in \mathbb{F}_q$ used in Pedersen commitments and note derivations to render commitments information-theoretically hiding.
- **Bonding Curve**: A mathematical formula ($x \cdot y = k$) defining the continuous relationship between token supply and token price.
- **Bulletproofs++**: An optimized zero-knowledge range proof argument eliminating trusted setups and yielding logarithmic proof sizes.

C

- **Circuit Breaker**: An on-chain emergency pause mechanism (`setPaused`) enabling the Guardian Council to freeze contract operations during active exploits.
- **CLSAG (Compact Linkable Spontaneous Anonymous Group)**: The compact ring signature algorithm historically used in Monero transactions.
- **Commitment**: A cryptographic hash binding a user to a specific value without revealing that value.
- **Confidential Storage Slot**: An encrypted EVM storage slot whose plaintext value is inaccessible to public observers without zero-knowledge authorization.

D

- **Dandelion++**: A peer-to-peer transaction propagation protocol featuring stem and fluff phases to prevent network surveillance nodes from triangulating the origin IP.

- **DKSAP (Dual-Key Stealth Address Protocol)**: A cryptographic protocol allowing senders to derive unlinkable one-time public keys using a recipient's static view and spend keys.

E

- **Enote (Electronic Note)**: A modular transaction output object separating ownership proof, value commitment, and nullifier generation.

F

- **FCMP++ (Full-Chain Membership Proofs)**: A cryptographic advancement replacing small ring signatures with zero-knowledge membership proofs across the entire ledger history.
- **Fixed Pool**: A launchpad model where tokens are distributed at an unchanging fixed rate until the supply cap is exhausted.

G

- **Gas Abstraction**: The capability for users to pay EVM transaction execution fees in arbitrary tokens (SER, XMR) without holding native ETH.
- **Groth16**: A zero-knowledge SNARK proof system producing constant-size 128-byte proofs verifiable on Ethereum with minimal gas overhead.

H

- **Homomorphic Addition**: An algebraic property where operating on encrypted commitments yields a valid commitment to the sum of the plaintexts: $(v_1 H + r_1 G) + (v_2 H + r_2 G) = (v_1 + v_2)H + (r_1 + r_2)G$.

I

- **Incoming View Key**: A private scalar key allowing a wallet to scan public blockchain blocks and decrypt incoming transaction amounts and memos without granting spending authority.
- **Incremental Merkle Tree**: A specialized append-only Merkle tree structure allowing logarithmic insertion ($O(\log N)$) and constant on-chain storage.

J

- **Jamtis**: The comprehensive wallet addressing standard designed to accompany the Seraphis enote protocol.

K

- **Key Image**: A deterministic cryptographic tag derived from a private key and its corresponding public key ($I = x \mathcal{H}_p(P)$) used to prevent double-spending in ring signature schemes.

M

- **MEV (Maximal Extractable Value)**: Profit extracted by miners, validators, or searchers by arbitrarily including, excluding, or reordering transactions in a block.
- **Mempool**: The memory pool where pending transactions wait prior to block confirmation.

N

- **Nullifier**: A deterministic 32-byte cryptographic identifier derived from a secret spend key and note index ($\mathcal{N} = \text{Keccak256}(\text{spendKey} \mathbin{\Vert} \text{leafIndex})$). When revealed, it proves a note has been spent without revealing which note it was.

O

- **Outgoing View Key**: A private scalar key authorizing the generation of verifiable tax compliance and audit disclosure packages.

P

- **Pedersen Commitment**: A computationally binding and information-theoretically hiding commitment scheme defined by $C = vH + rG$.
- **Privacy Continuum**: The four-tier privacy spectrum of Seraph spanning Level 0 (public EVM) to Level 3 (confidential contract state).
- **Pull-Escrow**: A defensive smart contract design pattern where balances accumulate in internal accounting records and are claimed by recipients, preventing malicious fallback revert attacks.

R

- **Range Proof**: A zero-knowledge proof establishing that a confidential amount v lies within a valid non-negative interval $[0, 2^{64}-1]$ without revealing v.
- **R1CS (Rank-1 Constraint System)**: A system of quadratic equations used to represent computational circuits in zero-knowledge SNARKs.

S

- **Sell Tax**: A 1.00% protocol fee assessed exclusively on sell transactions of SER tokens, split 50% to a permanent burn address and 50% to the Protocol Treasury.
- **Seraph EVM**: The general-purpose confidential EVM where transaction gas is settled in SER tokens.
- **Shielding**: Depositing public ERC-20 tokens into the `PrivacyPool` to mint a confidential private note.
- **Stealth Address**: A one-time public key derived mathematically such that only the designated recipient can compute the corresponding private spend key.

T

- **Tax Exemption Mapping (`taxExempt`)**: An on-chain registry allowing governance to whitelist DEX routers (Uniswap v3, Aerodrome) and CEX custody wallets from the 1.00% sell tax.
- **Threshold Cryptography**: A cryptographic scheme where a private key is split across n parties, requiring a quorum of t participants to decrypt data or sign messages.

U

- **Unshielding**: Withdrawing private notes from the `PrivacyPool` to a public Ethereum recipient address with zero protocol tax.
- **Usage Ticket**: A one-time cryptographic voucher consumed by autonomous AI agents to pay compute providers without linking payments to master wallets.

W

- **wXMR**: An ERC-20 token representing bridged Monero on EVM chains with 1:1 backing in base-layer multi-sig vaults.

X

- **XMR EVM**: A specialized EVM environment where transaction gas fees are paid in native XMR or `wXMR`.
- **xUSD**: A confidential, decentralized stablecoin minted against over-collateralized SER and XMR deposits via the `CollateralVault`.

Chapter 21: Contributing, Standards & Dev Setup

Chapter 21: Contributing, Standards & Dev Setup

21.1 Development Environment Prerequisites

To contribute to Seraph Protocol, ensure your local development workstation is configured with the following toolchains:

1. **Foundry**: Ethereum smart contract development framework.
   ```
   curl -L https://foundry.paradigm.xyz | bash
   foundryup
   ```
2. **Rust & Cargo**: Latest stable toolchain (1.80+).
   ```
   curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh
   ```
3. **Node.js**: Modern JavaScript runtime (Node.js v20+ or v22+).
4. **Git**: Distributed version control system.

21.2 Repository Setup & Verification

Clone the monorepo and verify all build targets:

```
# Clone the repository
git clone https://github.com/seraphprotocol/seraph.git
cd seraph

# 1. Verify and compile Foundry smart contracts
export PATH="$HOME/.foundry/bin:$PATH"
forge build
forge test

# 2. Verify and compile the Rust base-layer node
```

```
cargo build --manifest-path base-layer/Cargo.toml
cargo test --manifest-path base-layer/Cargo.toml

# 3. Verify TypeScript SDK & UI test suites
node --test sdk/typescript/test/*.test.mjs
```

21.3 Coding Standards & Testing Requirements

All contributions to Seraph are evaluated against strict architectural guidelines:

1. Smart Contract Standards (Solidity)

- Use Solidity `^0.8.24` with EVM target `cancun`.
- Inherit from OpenZeppelin v5 contracts.
- All public external functions accepting external asset transfers must utilize `nonReentrant`.
- All fee claims and external transfers must follow the **pull-over-push** design pattern.
- Any new contract modification must include corresponding Foundry unit tests and fuzz invariant tests.

2. Base Layer Standards (Rust)

- Adhere to standard Rust formatting: `cargo fmt --check`.
- Zero warnings from `cargo clippy`.
- Any modification to cryptographic commitments or nullifier hashing must be accompanied by multi-asset isolation tests in `base-layer/tests/adversarial_pentest.rs`.

3. Frontend & SDK Standards (TypeScript & Web)

- Zero external runtime framework bloat where possible.
- All on-chain string data injected into the DOM must be sanitized via `lpEsc()` to prevent XSS vulnerabilities.
- Cryptographic keys must remain in memory as typed byte arrays (`Uint8Array`) and never logged to console or local storage unencrypted.

21.4 Submitting Pull Requests & Git Conventions

1. **Branch Naming**:
   - `feat/feature-name` for new protocol capabilities.
   - `fix/bug-description` for security or logic patches.

- `docs/documentation-update` for documentation improvements.

2. **Commit Messages**: Follow the Conventional Commits specification:

```
feat(core): add taxExempt mapping for DEX routers
fix(pool): attach whenNotPaused to shield and unshield
test(pentest): add multi-actor invariant tests for gas relayer
docs(bookstack): add Chapter 18 on emergency procedures
```

3. **Pull Request Quality Gate**:
   - Every PR must pass all 155+ Foundry tests with 0 reverts.
   - Every PR must pass all 17 Rust base-layer pentests.
   - Every PR must pass all 43 Node.js verification tests.
   - Two approvals from core maintainers are required prior to merging into `main`.